

Module: `tf.compat.v1.flags` Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/flags

Import router for `absl.flags`. See <https://github.com/abseil/abseil-py>

Documentation

Import router for `absl.flags`. See <https://github.com/abseil/abseil-py>

Classes

class `ArgumentParser`: Base class used to parse and convert arguments.

class `ArgumentSerializer`: Base class for generating string representations of a flag value.

class `BaseListParser`: Base class for a parser of lists of strings.

class `BooleanFlag`: Basic boolean flag.

class `BooleanParser`: Parser of boolean values.

class `CantOpenFlagFileError`: Raised when flagfile fails to open.

class `CsvListSerializer`: Base class for generating string representations of a flag value.

class `DuplicateFlagError`: Raised if there is a flag naming conflict.

class EnumClassFlag: Basic enum flag; its value is an enum class's member.

class EnumClassListSerializer: A serializer for :class:MultiEnumClass flags.

class EnumClassParser: Parser of an Enum class member.

class EnumClassSerializer: Class for generating string representations of an enum class flag value.

class EnumFlag: Basic enum flag; its value can be any string from list of enum_values.

class EnumParser: Parser of a string enum value (a string value from a given set).

class Error: The base class for all flags errors.

class Flag: Information about a command-line flag.

class FlagHolder: Holds a defined flag.

class FlagNameConflictsWithMethodError: Raised when a flag name conflicts with :class:FlagValues methods.

class FlagValues: Registry of :class:~absl.flags.Flag objects.

class FloatParser: Parser of floating point values.

class IllegalFlagValueError: Raised when the flag command line argument is illegal.

class IntegerParser: Parser of an integer value.

class ListParser: Parser for a comma-separated list of strings.

class ListSerializer: Base class for generating string representations of a flag value.

class MultiEnumClassFlag: A multi_enum_class flag.

class MultiFlag: A flag that can appear multiple time on the command-line.

class UnparsedFlagAccessError: Raised when accessing the flag value from unparsed :class:FlagValues.

class UnrecognizedFlagError: Raised when a flag is unrecognized.

class ValidationError: Raised when flag validator constraint is not satisfied.

class WhitespaceSeparatedListParser: Parser for a whitespace-separated list of strings.

Functions

DEFINE(...): Registers a generic Flag object.

DEFINE_alias(...): Defines an alias flag for an existing one.

DEFINE_bool(...): Registers a boolean flag.

DEFINE_boolean(...): Registers a boolean flag.

DEFINE_enum(...): Registers a flag whose value can be any string from enum_values.

DEFINE_enum_class(...): Registers a flag whose value can be the name of enum members.

DEFINE_flag(...): Registers a :class:Flag object with a :class:FlagValues object.

DEFINE_float(...): Registers a flag whose value must be a float.

DEFINE_integer(...): Registers a flag whose value must be an integer.

DEFINE_list(...): Registers a flag whose value is a comma-separated list of strings.

DEFINE_multi(...): Registers a generic MultiFlag that parses its args with a given parser.

DEFINE_multi_enum(...): Registers a flag whose value can be a list strings from enum_values.

DEFINE_multi_enum_class(...): Registers a flag whose value can be a list of enum members.

DEFINE_multi_float(...): Registers a flag whose value can be a list of arbitrary floats.

DEFINE_multi_integer(...): Registers a flag whose value can be a list of arbitrary integers.

DEFINE_multi_string(...): Registers a flag whose value can be a list of any strings.

DEFINE_spaceselist(...): Registers a flag whose value is a whitespace-separated list of strings.

DEFINE_string(...): Registers a flag whose value can be any string.

FLAGS(...): Registry of :class:`~absl.flags.Flag` objects.

adopt_module_key_flags(...): Declares that all flags key to a module are key to the current module.

declare_key_flag(...): Declares one flag as key to the current module.

disclaim_key_flags(...): Declares that the current module will not define any more key flags.

doc_to_help(...): Takes a doc string and reformats it as help.

flag_dict_to_args(...): Convert a dict of values into process call parameters.

get_help_width(...): Returns the integer width of help lines that is used in TextWrap.

mark_bool_flags_as_mutual_exclusive(...): Ensures that only one flag among flag_names is True.

`mark_flag_as_required(...)`: Ensures that flag is not None during program execution.

`mark_flags_as_mutual_exclusive(...)`: Ensures that only one flag among `flag_names` is not None.

`mark_flags_as_required(...)`: Ensures that flags are not None during program execution.

`multi_flags_validator(...)`: A function decorator for defining a multi-flag validator.

`override_value(...)`: Overrides the value of the provided flag.

`register_multi_flags_validator(...)`: Adds a constraint to multiple flags.

`register_validator(...)`: Adds a constraint, which will be enforced during program execution.

`set_default(...)`: Changes the default value of the provided flag object.

`text_wrap(...)`: Wraps a given text to a maximum line length and returns it.

`validator(...)`: A function decorator for defining a flag validator.